KZG10 다항식 약정 체계 분석

1. KZG10 약정 체계 개요

KZG10(Kate-Zaverucha-Goldberg) 다항식 약정 체계는 PLONK에서 사용되는 주요 다항식 약정 체계 중 하나입니다. 이 약정 체계는 타원 곡선 페어링(pairing)을 사용하여 다항식에 대한 상수 크기의 약정을 생성하고 검증하는 방법을 제공합니다.

KZG10은 다음과 같은 특성을 가집니다:

- **상수 크기 약정**: 다항식의 차수에 관계없이 약정의 크기는 상수입니다.
- **상수 크기 증명**: 다항식의 특정 지점에서의 평가 증명도 상수 크기입니다.
- **동형성(Homomorphism)**: 다항식 약정의 선형 결합은 약정의 선형 결합과 같습니다.

2. PLONK에서의 KZG10 구현

PLONK 코드베이스에서 KZG10은 `ark_poly_commit` 크레이트의 `SonicKZG10` 을 기반으로 구현되어 있습니다. 이 구현은 `HomomorphicCommitment` 트레이트를 통해 확장되어 있습니다.

```
/// The Default KZG-style commitment scheme

pub  type KZG10<E> = SonicKZG10<E, DensePolynomial<<E as PairingEngine>::Fr>>;

/// A single KZG10 commitment

pub  type KZG10Commitment<E> = <KZG10<E> as PolynomialCommitment<

<E as PairingEngine>::Fr,

DensePolynomial<<E as PairingEngine>::Fr>,

>>::Commitment;
```

여기서 `E` 는 페어링 엔진 타입으로, 주로 `Bls12_381` 이나 `Bls12_377` 과 같은 타입이 사용됩니다.

3. HomomorphicCommitment 트레이트

KZG10은 `HomomorphicCommitment` 트레이트를 구현합니다. 이 트레이트는 다항식 약정의 동형성을 활용하기 위한 메서드를 제공합니다.

```
/// A homomorphic polynomial commitment
pub trait HomomorphicCommitment<F>:
PolynomialCommitment<F, DensePolynomial<F>>
where
F: PrimeField,
Self::VerifierKey: core::fmt::Debug,
{
/// Combine a linear combination of homomorphic commitments
fn multi_scalar_mul(
commitments: &[Self::Commitment],
scalars: &[F],
) -> Self::Commitment;
}
```

`multi_scalar_mul` 메서드는 여러 약정의 선형 결합을 계산합니다. 이는 다항식의 선형 결합에 대한 약정을 효율적으로 계산하는 데 사용됩니다.

4. KZG10의 구현 세부 사항

KZG10의 `HomomorphicCommitment` 구현은 다음과 같습니다:

```
impl<E> HomomorphicCommitment<E::Fr> for KZG10<E>
where
E: PairingEngine,
{
fn multi_scalar_mul(
commitments: &[KZG10Commitment<E>],
scalars: &[E::Fr],
) -> KZG10Commitment<E> {
let scalars_repr = scalars
.iter()
.map(<E::Fr as PrimeField>::into_repr)
.collect::<Vec<_>>();

let points_repr = commitments.iter().map(|c| c.0).collect::<Vec<_>>();

ark_poly_commit::kzg10::Commitment::<E>(
VariableBaseMSM::multi_scalar_mul(&points_repr, &scalars_repr)
.into(),
)
}
}
```

이 구현은 `VariableBaseMSM::multi_scalar_mul` 을 사용하여 다중 스칼라 곱셈(Multi-Scalar Multiplication)을 수행합니다. 이는 타원 곡선 포인트의 선형 결합을 효율적으로 계산하는 방법입니다.

5. KZG10의 수학적 기반

KZG10은 다음과 같은 수학적 원리를 기반으로 합니다:

5.1 약정 생성

다항식 $f(X)$에 대한 약정을 생성하기 위해, 신뢰할 수 있는 설정(trusted setup)에서 생성된 $\{g, g^s, g^{s^2}, \ldots, g^{s^d}\}$ 값을 사용합니다. 여기서 g는 타원 곡선의 생성자이고, s는 비밀 값입니다.

약정 C는 다음과 같이 계산됩니다:

$$C = f(s) \cdot g = \sum_{i=0}^{d} f_i \cdot g^{s^i}$$

여기서 f_i는 다항식 $f(X)$의 i번째 계수입니다.

5.2 평가 증명

다항식 $f(X)$가 특정 지점 z에서 $f(z) = y$라는 것을 증명하기 위해, 증명자는 다음과 같은 증명을 생성합니다:

1. $q(X) = \frac{f(X)-y}{X-z}$ 다항식을 계산합니다.
2. $\pi = q(s) \cdot g$를 계산하여 증명으로 제출합니다.

검증자는 다음 등식을 확인하여 증명을 검증합니다:

$$e(C - y \cdot g, h) = e(\pi, h^s / h^z)$$

여기서 e는 페어링 연산이고, h는 다른 그룹의 생성자입니다.

6. KZG10의 장단점

6.1 장점

1. **상수 크기 약정과 증명**: 다항식의 차수에 관계없이 약정과 증명의 크기가 상수입니다.
2. **효율적인 검증**: 검증 과정이 매우 효율적이며, 상수 시간에 수행됩니다.
3. **동형성**: 다항식의 선형 결합에 대한 약정을 효율적으로 계산할 수 있습니다.

6.2 단점

1. **신뢰할 수 있는 설정 필요**: KZG10은 신뢰할 수 있는 설정 과정이 필요하며, 이 과정에서 생성된 비밀 값 s가 누출되면 보안이 손상됩니다.
2. **페어링 연산 필요**: 검증 과정에서 페어링 연산이 필요하며, 이는 계산 비용이 높을 수 있습니다.
3. **양자 컴퓨팅에 취약**: KZG10은 이산 로그 문제에 기반하므로, 양자 컴퓨팅에 취약할 수 있습니다.

7. PLONK에서의 KZG10 활용

PLONK에서 KZG10은 다음과 같은 용도로 활용됩니다:

1. **다항식 약정**: 증명자는 와이어 다항식, 순열 다항식, 몫 다항식 등에 대한 약정을 생성합니다.
2. **배치 증명**: 여러 다항식의 평가를 한 번에 증명하기 위해 배치 기법을 사용합니다.
3. **선형 결합**: 여러 다항식의 선형 결합에 대한 약정을 효율적으로 계산합니다.

```
/// Computes a linear combination of the polynomial evaluations and polynomial

/// commitments provided a challenge.

pub fn linear_combination<F, H>(

evals: &[F],

commitments: &[H::Commitment],

challenge: F,

) -> (H::Commitment, F)
```

```
where

F: PrimeField,

H: HomomorphicCommitment<F>,

{

assert_eq!(evals.len(), commitments.len());

let  powers = crate::util::powers_of(challenge)

.take(evals.len())

.collect::<Vec<_>>();

let  combined_eval = evals

.iter()

.zip(powers.iter())

.map(|(&eval, power)| eval * power)

.sum();

let  combined_commitment = H::multi_scalar_mul(commitments, &powers);

(combined_commitment, combined_eval)

}
```

이 함수는 다항식 평가값과 약정의 선형 결합을 계산합니다. 이는 PLONK의 증명 생성 및 검증 과정에서 중요한 역할을 합니다.

8. KZG10의 보안 속성

8.1 계산적 바인딩 (Computational Binding)

KZG10의 바인딩 속성은 이산 로그 가정(Discrete Logarithm Assumption)에 기반합니다. 공격자가 동일한 약정에 대해 서로 다른 두 개의 다항식을 찾는 것은 계산적으로 불가능합니다.

8.2 완벽한 은닉 (Perfect Hiding)

KZG10은 완벽한 은닉 속성을 제공하지 않습니다. 약정 자체가 다항식에 대한 정보를 노출할 수 있습니다. 이를 해결하기 위해 블라인딩 팩터(blinding factor)를 사용할 수 있습니다.

8.3 신뢰할 수 있는 설정의 중요성

KZG10의 보안은 신뢰할 수 있는 설정 과정에서 생성된 비밀 값 s가 안전하게 폐기되는 것에 의존합니다. 만약 이 값이 노출되면:

- 공격자가 임의의 다항식에 대한 유효한 증명을 생성할 수 있습니다.
- 전체 시스템의 보안이 손상됩니다.

9. PLONK에서의 KZG10 성능 특성

9.1 벤치마크 결과

PLONK 코드베이스의 벤치마크에 따르면, KZG10은 다음과 같은 성능 특성을 보입니다:

```
fn  kzg10_benchmarks(c: &mut Criterion) {
constraint_system_benchmark::<
<Bls12_381 as PairingEngine>::Fr,
EdwardsParameters,
KZG10<Bls12_381>,
>("KZG10", c);
}
```

9.2 메모리 사용량

KZG10 약정의 크기는 타원 곡선 포인트 하나의 크기와 같으며, 이는 일반적으로 32-48 바이트입니다. 이는 다항식의 차수에 관계없이 일정합니다.

9.3 계산 복잡도

- **약정 생성**: O(n) - 다항식의 차수에 비례
- **증명 생성**: O(n) - 다항식의 차수에 비례
- **검증**: O(1) - 상수 시간

10. 실제 사용 예제

PLONK 코드베이스의 예제에서 KZG10이 어떻게 사용되는지 살펴보겠습니다:

```
// examples/simple_circuit.rs에서
type PC = SonicKZG10<Bls12_381, DensePolynomial<BlsScalar>>;
let  pp = PC::setup(1 << 10, None, &mut OsRng)
.map_err(to_pc_error::<BlsScalar, PC>)?;
```

이 예제에서는:

1. `Bls12_381` 페어링 엔진을 사용합니다.
2. `1 << 10` (1024)개의 제약 조건을 지원하는 설정을 생성합니다.
3. 랜덤 시드를 사용하여 신뢰할 수 있는 설정을 생성합니다.

11. 결론

KZG10은 PLONK에서 사용되는 효율적인 다항식 약정 체계입니다. 상수 크기의 약정과 증명, 효율적인 검증, 동형성 등의 특성으로 인해 영지식 증명 시스템에 적합합니다. 그러나 신뢰할 수 있는 설정이 필요하고 페어링 연산의 계산 비용이 높다는 단점이 있습니다.

PLONK 코드베이스에서 KZG10은 `HomomorphicCommitment` 트레이트를 통해 구현되어 있으며, 이를 통해 다항식 약정의 동형성을 활용할 수 있습니다. 이러한 구현은 PLONK의 효율성과 보안성을 높이는 데 기여합니다.

KZG10의 보안은 이산 로그 가정과 신뢰할 수 있는 설정의 안전성에 의존하며, 실제 사용에서는 이러한 요소들을 신중히 고려해야 합니다.